

Time Will Tell: Criss-Cross Transformer for Encrypted Traffic Analysis

Hua Ding , *Student Member, IEEE*, Lixing Chen , Bo Zhang, Shenghong Li , *Senior Member, IEEE*, Hao Peng , Zhe Qu , *Member, IEEE*, and Yang Bai , *Member, IEEE*

Abstract—The widespread adoption of encryption across web-based services is compelling both malicious attackers and network defenders to tailor their tool repositories to encrypted traffic. For various security applications in encrypted networks, the analysis of encrypted traffic lies as the fundamental basis. Due to the inherent concealment of content-related information in encrypted packets, the dynamics of encrypted traffic emerge as the discernible variable warranting comprehensive analysis. This paper explores inherent temporal correlations within the encrypted traffic and proposes a novel algorithm called **Criss-cross Traffic Transformer (CTT)**, tailored to address unique challenges in encrypted traffic analysis. CTT distinguishes itself by employing a specialized time series Transformer that innovatively utilizes *patching* and *criss-cross attention module* (CAM) to dissect and interpret encrypted traffic, with the “criss” part mining the long-/short-term temporal correlations across time, and the “cross” part capturing temporal correlations across multiple feature dimensions of encrypted traffic. CTT provides a unified framework capable of accommodating diverse analytical granularities, including packet-level, flow-level, and packet-to-flow level. Notably, CTT not only encompasses encrypted traffic classification but also extends to encrypted traffic forecasting, an area that remains largely underexplored in existing literature. We evaluate CTT in the context of fingerprinting attacks and malware detection over 5 real-world datasets against 13 benchmarks. The results indicate that CTT achieves up to 15.56% performance improvement over SOTA solutions for encrypted traffic classification. Particularly, CTT demonstrates over 92.5% forecasting accuracy, which is comparable to SOTA performances in

the seen-and-classify scenario, suggesting its potential applicability to broader domains like social network behavioral analysis.

Index Terms—Encrypted traffic classification, encrypted traffic forecasting, time-series transformer, cross-time cross-dimension attention.

I. INTRODUCTION

AMIDST escalating cyber threats, encrypted Internet traffic has become essential for safeguarding sensitive data against unauthorized access and interception. As reported in [1], over 80% of Firefox services and 95% of Google services are now encrypted, ensuring the secure transmissions of confidential information and enhancing user privacy, particularly for user interactions on social network platforms. On the other hand, traffic encryption is also misused by cybercriminals to obfuscate malicious activities. Empirical evidence [2] indicates that 85% of cyberattacks are conducted via encrypted traffic. For example, encryption is often employed for propagating commands and payloads of malware, and is also frequently seen in data exfiltration activities where stolen intellectual property is encrypted to evade detection by data loss prevention measures. With the prevalence of encryption on the Internet, both malicious attackers and network defenders are tailoring their tool repositories to encrypted traffic. From the perspective of malicious attackers, novel cyberattacks are emerging specifically designed to target encrypted traffic. For example, website/application fingerprinting attacks [3] have been developed to compromise user privacy by inferring service usage behavior (e.g., identifying specific social apps) through the analysis of encrypted traffic. From the perspective of network defenders, developing sophisticated defense tools to counter cyberattacks with encryption has become imperative. For example, intrusion detection [4], [5] and malware detection methods [6] are increasingly shifting their focus from non-encrypted traffic to encrypted traffic.

Upon removing extraneous layers, the majority of the aforementioned security applications in encrypted networks essentially pertain to problems of encrypted traffic analysis [7]. Consequently, the efficacy of encrypted traffic analysis serves as a critical determinant of encrypted network security. Encrypted traffic analysis distinguishes itself from traditional traffic analysis methods [8], [9] by the capability to operate despite the concealment of content-related information in encrypted packets. Early studies of encrypted traffic analysis [3] leverage residual plaintext in encrypted traffic to construct fingerprints for analysis. Nevertheless, these approaches demonstrate limited efficacy in practice due to the sparse availability of plaintext. Traditional machine learning (ML) techniques like Support Vector

Received 13 August 2025; revised 6 December 2025; accepted 9 February 2026. Date of publication 13 February 2026; date of current version 10 April 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3107100, in part by the National Natural Science Foundation of China (NSFC) under Grant 62372297, Grant 62303306, and Grant 62302525, in part by China Postdoctoral Science Foundation under Grant 2025M771527, and in part by the Hunan Province Natural Science Foundation of China under Grant 2024JJ6527. (Corresponding authors: Lixing Chen; Bo Zhang; Shenghong Li.)

Hua Ding, Lixing Chen, and Shenghong Li are with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China, and also with Shanghai Key Laboratory of Integrated Administration Technologies for Information Security, Shanghai 200240, China (e-mail: dinghua623@sjtu.edu.cn; lxchen@sjtu.edu.cn; shli@sjtu.edu.cn).

Bo Zhang is with China Electric Power Research Institute, Beijing 100192, China (e-mail: zhangbo6@epri.sgcc.com.cn).

Hao Peng is with the School of Computer Science and Technology, Zhejiang Normal University, Jinhua 321004, China (e-mail: hpeng@zjnu.edu.cn).

Zhe Qu is with the School of Computer Science and Engineering, Central South University, Changsha 410083, China (e-mail: zhe_qu@csu.edu.cn).

Yang Bai is with the School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: ybai@sjtu.edu.cn).

Our code is available at https://github.com/Amanda-HuaDing/Criss-cross_Transformer.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2026.3664705>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2026.3664705

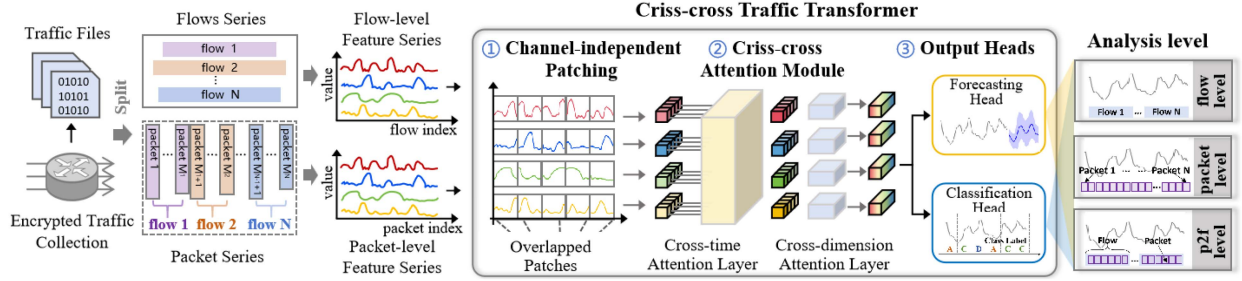


Fig. 1. Overview of criss-cross traffic transformer for encrypted traffic analysis.

Classifier (SVC) and Random Forest have also been employed for encrypted traffic analysis [10], [11], achieving reasonable accuracy in traffic classification tasks. However, their heavy reliance on handcrafted features restricts their generalization capabilities. Deep learning (DL) techniques [12], [13], owing to their exceptional capability of automatic feature extraction, have become the primary focus in encrypted traffic analysis. Given the concealment of content-related information in encrypted traffic, the temporal correlations and dynamics of encrypted traffic represent one of the few analyzable variables. State-of-the-art methods [14], [15] utilize Recurrent Neural Networks (RNNs) [13] and Long Short-Term Memory (LSTM) [16] to generate the temporal insights within the encrypted traffic. While these methods demonstrate success in capturing local correlations, they struggle to identify long-term temporal dependencies due to their limited scope of adjacent traffic. Besides, the majority of these state-of-the-art approaches are confined to classification paradigms—assigning labels to observed traffic retrospectively. This “detect-after-the-fact” nature is inherently reactive and fails to anticipate evolving traffic states.

To illustrate the temporal correlations in encrypted traffic, we conduct a case study on real-world data, with empirical observations detailed in Section V-B. The results reveal both short- and long-term temporal correlations within univariate feature series and across multiple feature dimensions. These temporal correlations provide critical insights into regular activities and anomalies in encrypted networks, making their understanding essential for interpreting security issues.

In response to the above observations, this paper introduces a novel algorithm named Criss-Cross Traffic Transformer (CTT) for encrypted traffic analysis in security applications. An overview of CTT is depicted in Fig. 1. CTT is a specialized time series Transformer [17], [18] tailored to efficiently capture both short-term and long-term temporal correlations within encrypted traffic. Besides, CTT is equipped with the capability to explore and extract time dependencies across feature dimensions, thereby enhancing analytical performance for encrypted traffic with multiple feature variates. CTT is designed to deliver a unified framework that accommodates various analytical levels for encrypted traffic analysis, including packet-level, flow-level, and packet-to-flow (p2f) level, offering generalized and granularity-adaptable solutions for diverse security applications. Particularly noteworthy is that CTT presents an innovative long-term forecasting functionality for encrypted traffic by leveraging its exceptional capability of capturing long-term temporal correlations. To the best of our knowledge, this paper makes an initial endeavor towards forecasting encrypted traffic patterns. The contributions of this work are summarized as follows:

1) *Criss-cross Traffic Transformer (CTT) for Encrypted Traffic Analysis*: CTT first performs channel-independent patching to segment each feature series of encrypted traffic into overlapping patches. These patches are then fed to the Criss-cross Attention Module (CAM), a pivotal component of CTT. The “criss” part of CAM identifies short-/long-term temporal correlations within individual feature series, and the “cross” part of CAM captures temporal correlations across multiple feature series of encrypted traffic. CTT converts multivariate feature series of encrypted traffic to feature representations enriched with temporal insights, which are subsequently utilized for completing downstream tasks.

2) *Unified Classification and Forecasting Framework for Encrypted Traffic*: We investigate classification and forecasting tasks for encrypted traffic based on CTT. The classification task involves predicting a label series for encrypted traffic, while the forecasting task aims to forecast a sequence of labels for forthcoming encrypted traffic based on currently observed traffic. Particularly, we emphasize long-term forecasting scenarios, where the forecasted sequence extends beyond 20 time steps. We develop dedicated components known as the classification head and forecasting head, which accomplish classification and forecasting tasks, respectively. Notably, classification and forecasting share the same upstream process and can be performed simultaneously.

3) *Comprehensive Multi-level Analysis and Systematic Evaluations*: CTT exhibits adaptability and generality across various analytical levels in encrypted traffic analysis. We cover flow level (i.e., analyzing entire traffic flows), packet level (i.e., analyzing individual packets), and packet-to-flow level (i.e., analyzing traffic flow using packet-level information) within a uniform framework. We systematically evaluate the performance of CTT on all 3 analytical levels, using 5 real-world datasets in the context of 2 security levels, and against 13 benchmarks. The experimental results show that CTT achieves up to 15.56% improvement over SOTA solutions for encrypted traffic classification. Additionally, CTT demonstrates over 92.5% forecasting accuracy, comparable to SOTA performances in seen-and-classify scenarios.

II. RELATED WORKS

Encrypted Traffic Analysis for Security Applications: Existing works have extensively investigated encrypted traffic analysis for security applications. These works can be categorized into three main domains: (1) website/application fingerprinting attack, (2) malicious attack detection, and (3) user privacy protection [19]. Studies in fingerprinting attacks focus on identifying

user activity [3], [10], thereby compromising the anonymity of encrypted networks. Works in malicious attack detection target network intrusion detection [4], [5] and malware detection [6], [20], safeguarding network security by uncovering potential attacks hidden within encrypted traffic. Meanwhile, studies in user privacy protection aim to monitor mobile application characteristics [21], [22], empowering users to control their personally identifiable information. In the context of industrial deployment, Anderson et al. [11] underscored the significant challenges arising from non-stationarity and label noise in practical encrypted malware detection. Moreover, Bernighausen et al. [23] emphasized the imperative of real-time classification capabilities to accommodate the high throughput characteristic of modern network infrastructures.

While prior methodologies have addressed various application scenarios, they predominantly focus on encrypted traffic classification. In contrast, the proposed CTT not only encompasses encrypted traffic classification tasks but also enables the exploration of encrypted traffic forecasting. Several studies have explored the task of traffic forecasting. For instance, Huo et al. [24] applied an enhanced Seasonal and Trend decomposition algorithm to decompose traffic data, employing LSTM to forecast future network volume. Furthermore, Fang et al. [25] integrated Graph Neural Networks (GNNs) with LSTM to extract spatiotemporal features from mobile network traffic, enhancing the accuracy of traffic volume prediction. Montieri et al. [26] concentrated on fine-grained, packet-level mobile traffic characteristics prediction, emphasizing the explainability of their models. However, CTT differs from these works in task definition and application scope. First, most existing works focus on predicting specific traffic features or traffic volume, whereas CTT is designed to forecast the labels of future traffic. Second, these works focus on mobile communication scenarios without specific emphasis on encrypted traffic.

ML Solutions for Encrypted Traffic Analysis: Traditional ML techniques have found extensive applications in encrypted traffic analysis. Taylor et al. [10] proposed Appscanner, employing statistical features to train an SVC for identifying Android apps. Ede et al. [3] proposed Flowprint, which leverages temporal correlations among destination-related features to achieve traffic classification through clustering. The efficiency of ML-based methods heavily depends on the quality of feature selection and necessitates judicious feature extraction and selection for different scenarios.

DL has gained popularity in encrypted traffic analysis due to its ability for autonomous feature extraction. Sirinam et al. [27] introduced DF, leveraging CNN to capture patterns in packet size and direction. To compensate for the loss of inter-variable information caused by encryption, RNN is used to capture temporal patterns in the traffic. Liu et al. [14] proposed FS-Net based on RNN to learn temporal dependencies from packet sequences. Lin et al. [28] proposed TSCRNN, leveraging a combination of CNN and stacked bidirectional LSTM. While RNN-based methods have shown performance improvement compared to CNN-based methods, they have limited capability to capture long-term time dependencies [29], suggesting that there is still room for improvement.

Transformers have shown great modeling ability for long-range dependencies in sequential data in Computer Vision (CV) [30] and Natural Language Processing (NLP) [31], and thus are appealing to encrypted traffic analysis. Recent advancements have utilized Transformer-based models, adapting language or vision processing techniques for traffic feature

TABLE I
COMPARISON WITH EXISTING METHODS

Methods		Analysis granularity			Pre-train	Forecast
		flow	packet	p2f		
ML-based	Appscanner [10]	✓	✓	×	Not Req.	×
	Flowprint [3]	✓	×	×	Not Req.	×
CNN/RNN-based	DF [27]	×	×	✓	Not Req.	×
	Deeppacket [34]	✓	✓	×	Not Req.	×
	FS-Net [14]	×	×	✓	Not Req.	×
	TSCRNN [28]	×	×	✓	Not Req.	×
Transf.-based	ET-BERT [15]	✓	✓	×	Required	×
	YaTC [32]	×	×	✓	Required	×
	AN-Net [33]	×	×	✓	Not Req.	×
	CTT (ours)	✓	✓	✓	Not Req.	✓

extraction. ET-BERT [15] interprets traffic bytes as linguistic tokens, pre-training a Transformer for traffic feature learning. Zhao et al. [32] model network traffic as gray images and adopt a vision Transformer to extract multi-level flow representations. Deng et al. [33] introduced AN-Net, a model that processes traffic features independently before integrating them into a robust unified representation via a self-attention module. While Transformer-based methods have demonstrated performance gains over earlier approaches, they typically rely on extensive pre-training procedures, which are both time-consuming and computationally demanding. Moreover, such methodologies are predominantly restricted to traffic classification paradigms and are inherently ill-suited for traffic forecasting. The absence of this predictive capability precludes the generation of early warnings essential for proactive network management and the preemption of advanced threats. Table I provides a qualitative comparison between CTT and existing approaches in terms of methodological characteristics.

III. ENCRYPTED TRAFFIC ANALYSIS

A. Preliminaries of Encrypted Traffic Analysis

As aforementioned, encrypted traffic analysis lies as the fundamental basis for various security applications in encrypted networks. Our work analyzes encrypted traffic with a specific focus on temporal dependencies, which exerts minimal prerequisites on the form of encrypted traffic, thereby offering broad applications across typical implementation scenarios. Here, we present two typical security applications based on encrypted traffic analysis from the perspectives of both malicious attackers (*fingerprinting attack*) and network defenders (*malware detection*). These two scenarios also align with the experiments carried out in Section V. The fingerprinting attack aims to identify the applications that the user is visiting in an encrypted connection. In this case, the malicious attacker eavesdrops on encrypted traffic between clients and the network services, and then utilizes the encrypted traffic analysis to identify the application service in use based on the collected encrypted traffic. In the context of malware detection, a network administrator collects encrypted traffic at gateways or routers and analyzes encrypted traffic to distinguish whether the encrypted traffic is issued by benign software or malware.

The above application scenarios can be described by a process of encrypted traffic analysis. The encrypted traffic analysis is carried out by a dedicated agent. This agent can be the malicious attacker in the fingerprinting attack or the network administrator in the malware detection scenario. The agent is tasked with two primary functions: (1) collecting encrypted packets during user

communications, and (2) pre-processing collected encrypted packets and performing the designed encrypted analysis. This agent is deployable at intelligent switches, routers, and gateways, which are typical sites for implementing traffic collection tools such as Libpcap [35], TCPdump [35], etc. The enhanced computational capabilities of these high-end edge devices also allow for the execution of DL computations. During packet collection, the agent runs the traffic collection tool to capture encrypted packets.

B. Analytical Levels/ Tasks for Encrypted Traffic

Upon acquiring encrypted packets, the traffic analysis agent pre-processes them with analyzers such as Wireshark [36], and Tranalyzer [37]. The processing of encrypted traffic can be adapted to various granularities contingent on the implementation scenario. Herein, we present three analytical levels, *packet-level*, *flow-level*, and *packet-to-flow level*, alongside two analytical tasks, namely classification and forecasting.

1) Analytical Levels:

• **Packet-level Traffic Analysis:** This level is highly granular, facilitating near-real-time analysis for encrypted traffic, which fits application scenarios like real-time intrusion detection. In the context of packet-level analysis, encrypted traffic is characterized as a sequence of consecutive packets (pkt), denoted by $\text{Traffic} = \{\text{pkt}_\tau\}_{\tau=1}^\infty$, and its processing centers on extracting packet features. Due to the encryption, observable features of encrypted packets are restricted. Common features include the source/destination IP, source/destination port, packet size, packet payload size, etc. The features associated with pkt_τ are encapsulated in a feature vector $\mathbf{x}_\tau^{\text{pkt}} \in \mathbb{R}^{M^{\text{pkt}}}$, where M^{pkt} is the number of extracted features. By applying feature extraction to each packet in $\{\text{pkt}_\tau\}_{\tau=1}^\infty$, a packet-level feature series $\{\mathbf{x}_\tau^{\text{pkt}}\}_{\tau=1}^\infty$ can be constructed. The packet-level analysis is conducted on the packet-level feature series, where each packet serves as a fundamental unit.

• **Flow-level Traffic Analysis:** This level analyzes encrypted traffic flows rather than individual packets to provide a higher-level overview of network activity, making it suitable for security applications such as fingerprinting attacks. In the context of flow-level analysis, the encrypted traffic is represented as a sequence of successive flows (fl), denoted by $\text{Traffic} = \{\text{fl}_\tau\}_{\tau=1}^\infty$. A flow is defined as a collection of packets sharing specific common attributes within a predefined time interval. Therefore, the features of flows often encompass statistics derived from the packets contained in the flow, including the packet count, maximum packet length, minimum interval time, etc. We let $\mathbf{x}_\tau^{\text{fl}} \in \mathbb{R}^{M^{\text{fl}}}$ denote the feature vector of fl_τ , where M^{fl} is the number of extracted features. By applying feature extraction to each flow in $\{\text{fl}_\tau\}_{\tau=1}^\infty$, we can obtain a flow-level feature series $\{\mathbf{x}_\tau^{\text{fl}}\}_{\tau=1}^\infty$, on which flow-level traffic analysis is performed. The fundamental unit in the flow-level analysis is traffic flow.

• **Packet-to-flow Level Traffic Analysis:** This approach facilitates examining encrypted traffic at the flow level by analyzing underlying packet-level attributes. It inspects fine-grained information to obtain a high-level pattern of encrypted traffic, which is advantageous in advanced threat detection systems for identifying sophisticated malware. The p2f-level analysis processes encrypted traffic in the unit of packets $\text{Traffic} = \{\text{pkt}_\tau\}_{\tau=1}^\infty$. The packet sequence is segmented into flows $\{(\text{pkt}_1, \dots, \text{pkt}_F), (\text{pkt}_{F+1}, \dots, \text{pkt}_{2 \times F}), \dots\}$, with F

indicating the length of the segmented flow. Following similar procedures in packet-level analysis, a packet-level feature series $\{\mathbf{x}_\tau^{\text{pkt}}\}_{\tau=1}^\infty$ can be constructed. The p2f-level analysis receives $\{\mathbf{x}_\tau^{\text{pkt}}\}_{\tau=1}^\infty$ as input and generates analytical results for flows $\{\text{fl}_j\}_{j=1}^\infty$, where $\text{fl}_j = (\text{pkt}_{(j-1) \times F + 1}, \dots, \text{pkt}_{j \times F})$ denotes j -th flow.

2) Analytical Tasks:

• **Encrypted Traffic Classification:** The classification task takes a finite feature series $\vec{\mathbf{x}} = \{\mathbf{x}_1, \dots, \mathbf{x}_L\}$ as input, where $\mathbf{x}_l$ is either a flow-level feature vector $\mathbf{x}_\tau^{\text{fl}}$ or a packet-level feature vector $\mathbf{x}_\tau^{\text{pkt}}$ and L denotes the length of the feature series. Notably, the feature vectors are continuous in time, i.e., with $\mathbf{x}_1 \leftarrow \mathbf{x}_t$, the l -th feature vector in $\vec{\mathbf{x}}$ satisfies $\mathbf{x}_l \leftarrow \mathbf{x}_{t+l-1}$. For flow-level and packet-level classification, the goal of classification is to assign a label series $\hat{\mathbf{y}} = \{\hat{y}_1, \dots, \hat{y}_L\}$ to the input feature series $\vec{\mathbf{x}}$. Here, each label $\hat{y}_l \in [1, 2, \dots, C]$ (C is the number of classes) represents the predicted class for the l -th flow/packet. The classification for p2f-level analysis follows a similar formulation but differs in the length of the predicted label series. Given the packet-level feature series $\vec{\mathbf{x}}$, p2f classification analyzes it in form of feature series segments, $\vec{\mathbf{x}} = \{(\mathbf{x}_1, \dots, \mathbf{x}_F), \dots, (\mathbf{x}_{L-F+1}, \dots, \mathbf{x}_L)\}$ and generates a predicted label series $\hat{\mathbf{y}} = \{\hat{y}_1, \dots, \hat{y}_{L/F}\}$, where $\hat{y}_f, f = 1, \dots, L/F$, denotes the predicted class of f -th segment (i.e., flow). This formulation has a mild implicit assumption that $L > F$ and $L = u \times F, u = 1, 2, \dots$.

• **Encrypted Traffic Forecasting:** Compared to classification, traffic forecasting has received limited attention, largely due to the lack of effective supporting techniques. Traditional traffic forecasting works [25], [38] focus on low-level metrics such as traffic volume or throughput. In contrast, our work aims to predict semantic-level labels (e.g., application categories, malware families), offering significant advantages for both proactive network defense and quality of service (QoS) enhancement. For example, in malware detection scenarios, forecasting that an upcoming flow sequence is likely associated with a specific malware family enables the early deployment of mitigation strategies—such as preemptively blocking IPs or ports, dynamically adjusting firewall rules, isolating vulnerable resources, or prioritizing forensic analysis of suspected endpoints—before the malicious activity fully manifests. In this context, we outline the inputs and outputs involved in forecasting tasks. The forecasting task also operates on a finite flow-level or packet-level feature series $\vec{\mathbf{x}} = \{\mathbf{x}_1, \dots, \mathbf{x}_L\}$, serving as input. The primary objective of forecasting is to predict a label series $\hat{\mathbf{y}} = \{\hat{y}_{L+1}, \dots, \hat{y}_{L+T}\}$ corresponding to forthcoming encrypted traffic (unseen) over T time steps. We call L the look-back length, and T the forecasting length. The forecasted label $\hat{y}_{L+t}$ can represent the class label of a packet in the packet-level analysis or the class label of a flow in flow-level analysis. To address the inherent challenge of unlabeled encrypted flows in real-world environments, CTT supports two workflows: direct label-free forecasting of future traffic solely from feature vectors $\vec{\mathbf{x}}$, or alternatively, first classifying the lookback window to obtain predicted labels $\{\hat{y}_1, \dots, \hat{y}_L\}$ which are then propagated through the forecasting pipeline to generate $\{\hat{y}_{L+1}, \dots, \hat{y}_{L+T}\}$.

IV. CRISS-CROSS TRAFFIC TRANSFORMER

Criss-cross Traffic Transformer (CTT) is a specialized time series transformer designed specifically for encrypted traffic.

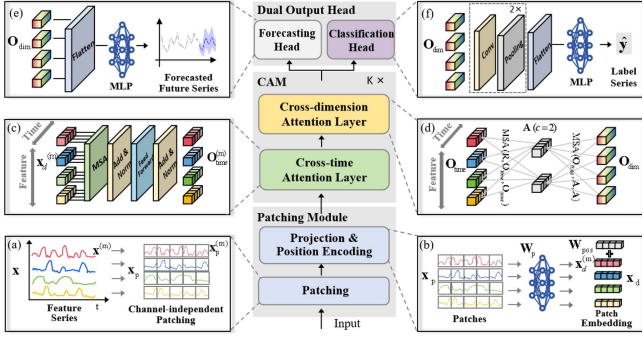


Fig. 2. Diagram of CTT architecture.

Fig. 2 presents an overview of CTT, which encompasses three key components: *Patching Module*, *Criss-cross Attention Module (CAM)*, and *Dual Output Head*.

Patching Module: The patching operation is a distinctive trait of CTT. Recall our objective is to understand the temporal correlations within encrypted traffic. However, an individual packet or flow offers restricted insight across time domain. Patching operations amalgamate time steps into subseries-level patches, thereby capturing comprehensive information that is not attainable through point-wise analysis. This approach enhances the locality of traffic data and facilitates the examination of short-term time correlations. Besides, patching enables the subsequent transformer to access longer historical traffic data, facilitating the analysis of long-term time correlations.

Criss-cross Attention Module: CAM is the core of CTT. Compared to the traditional Transformer, CAM utilizes a two-stage attention framework to efficiently capture the cross-time correlations and cross-dimension correlations. CTT comprises Cross-time Attention Layers (CTALs) and Cross-dimension Attention Layers (CDALs). The CTALs are designed to mine short-term and long-term temporal dependencies within patches of an individual feature series, whereas the CDALs capture the time dependencies across multiple feature series.

Dual Output Head: CTT incorporates two distinct output heads tailored to classification and forecasting tasks. The classification head utilizes a CNN-based structure to reconstruct temporal information from patch-level to point-level, thus enabling CTT to generate classification results of each data point (i.e., each packet/flow). The forecasting head utilizes a straightforward flattening structure to forecast future label series based on the temporal correlations observed in the lookback series. This dual output head makes CTT a robust solution for various encrypted traffic analysis scenarios.

A. Patching Module

The input feature series $\vec{\mathbf{x}} = \{\mathbf{x}_1, \dots, \mathbf{x}_L\}$ is a finite multi-variate series, where an element $\mathbf{x}_l \in \mathbb{R}^{M \times 1}$ in $\vec{\mathbf{x}}$ is the multi-dimension feature vector for a packet/flow. Therefore, $\vec{\mathbf{x}}$ can be decomposed into M univariate feature series. The m -th univariate feature series is denoted by $\vec{\mathbf{x}}^{(m)} = \{x_1^{(m)}, \dots, x_L^{(m)}\}$. To better mine the short-term and long-term dependencies within univariate feature series, CTT performs channel-independent patching. We employed an overlapped patching strategy, which generates feature patches based on the patch length P and stride S . Each subsequent patch is shifted backward by S steps relative

to the previous one, resulting in an overlap between consecutive patches of length $(P - S)$. For a univariate feature series $\vec{\mathbf{x}}^{(m)}$, the overlapped patching generates a sequence of N patch sequence $\mathbf{x}_p^{(m)} = \{x_{(n-1) \times S + 1}^{(m)}, \dots, x_{(n-1) \times S + P}^{(m)}\}_{n=1}^N$, where $N = \lfloor \frac{(L-P)}{S} \rfloor + 2$ is the total number of generated patches. To ensure that the length of the last patch is the same as the length of previous patches, we pad S repeated instances of the last value $x_L^{(m)} \in \mathbb{R}$ to the end of $\vec{\mathbf{x}}^{(m)}$. The patch length determines the granularity of the analysis. Longer patches capture broader temporal patterns but may miss finer details in brief intervals. Conversely, shorter patches enhance the resolution of analysis, detecting subtle variations but potentially overlooking wider trends. Additionally, the patch length and stride length jointly govern the overlap between patches. A larger patch overlap offers multiple contextual perspectives of the same data point across adjacent patches, thereby improving the capacity of CTT to learn nuanced features. The patch sequence $\mathbf{x}_p^{(m)}$ is then fed into Transformer, with each patch corresponding to a *token* (an individual unit of input data in Transformer). Patching reduces the number of input tokens from L to approximately L/S . This implies the computational complexity of attention maps is quadratically decreased by S .

The patch sequence $\mathbf{x}_p^{(m)}$ is mapped to the latent space of dimension d_{model} through a learnable linear projection $\mathbf{W}^p \in \mathbb{R}^{d_{\text{model}} \times P}$ and a learnable additive position encoding $\mathbf{W}^{pos} \in \mathbb{R}^{d_{\text{model}} \times N}$ for monitoring the temporal order of patches, which generates the embedding $\mathbf{x}_d^{(m)} \in \mathbb{R}^{d_{\text{model}} \times N}$ for patch sequence $\mathbf{x}_p^{(m)}$: $\mathbf{x}_d^{(m)} = \mathbf{W}^p \mathbf{x}_p^{(m)} + \mathbf{W}^{pos}$. The patch sequence embedding $\mathbf{x}_d^{(m)}$ serves as the input to CAM.

Why channel-independent patching? CTT utilizes channel-independent patching. An alternative strategy is channel-mixing patching [17], which aggregates all features in each time step into input tokens. However, this strategy may encounter challenges due to potential inconsistencies of the temporal dependencies among univariate feature series. Mixing these feature series during patching could obscure the temporal correlation within each univariate feature series. Therefore, we adopt the channel-independent patching strategy to better retain temporal correlations within each univariate feature series. The temporal correlations across multiple feature series will be addressed by the subsequent CAM module.

B. Criss-Cross Attention Module

Despite utilizing the channel-independent patching in CTT, it is noteworthy that the correlations among different feature series are not regarded as insignificant. To compensate for the information loss in cross-feature correlations, we propose the Criss-cross Attention Module (CAM). CAM strategically captures cross-time and cross-dimension temporal correlations through a two-stage process, which comprises two loosely coupled components Cross-Time Attention Layer (CTAL) and Cross-Dimension Attention Layer (CDAL). This sequential architecture is specifically designed to mitigate the computational complexity of CAM, as concurrent attention across both time and dimensions is computationally prohibitive.

Cross-time Attention Layer: CTAL directly applies the multi-head self-attention (MSA) mechanism [39] to capture temporal correlations within each feature series, as illustrated in Fig. 2 (c).

It takes patch embedding of univariate feature series $\mathbf{x}_d^{(m)}$, $m = 1, \dots, M$, as input. In the multi-head attention mechanism, each attention head, indexed by $h = 1, \dots, H$, transforms $\mathbf{x}_d^{(m)}$ into query matrices $\mathbf{Q}_h^{(m)} = (\mathbf{x}_d^{(m)})^\top \mathbf{W}_h^Q$, key matrices $\mathbf{K}_h^{(m)} = (\mathbf{x}_d^{(m)})^\top \mathbf{W}_h^K$, and value matrices $\mathbf{V}_h^{(m)} = (\mathbf{x}_d^{(m)})^\top \mathbf{W}_h^V$. Here, $\mathbf{W}_h^Q, \mathbf{W}_h^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_h^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, with $d_k = d_v = d_{\text{model}}/H$. Given the Query-Key-Value (QKV) matrices, each attention head utilizes the scaled dot-product attention to compute the attention scores, expressed as

$$\begin{aligned} (\mathbf{O}_h^{(m)})^\top &= \text{attention}(\mathbf{Q}_h^{(m)}, \mathbf{K}_h^{(m)}, \mathbf{V}_h^{(m)}) \\ &= \text{softmax} \left(\frac{\mathbf{Q}_h^{(m)} (\mathbf{K}_h^{(m)})^\top}{\sqrt{d_k}} \right) \mathbf{V}_h^{(m)}, \end{aligned} \quad (1)$$

where $\mathbf{O}_h^{(m)} \in \mathbb{R}^{d_k \times N}$ is the output of the h -th attention head. The output of MSA can be represented as Eqn. (2), where $\mathbf{W}^O$ is a learnable linear projection in MSA.

$$\begin{aligned} (\mathbf{O}^{(m)})^\top &= \text{MSA}_{\text{time}}(\mathbf{Q}^{(m)}, \mathbf{K}^{(m)}, \mathbf{V}^{(m)}) \\ &= \text{concat} \left((\mathbf{O}_1^{(m)})^\top, \dots, (\mathbf{O}_H^{(m)})^\top \right) \mathbf{W}^O. \end{aligned} \quad (2)$$

All feature dimensions share the same MSA_{time} . The multi-head attention block also includes LayerNorm layers and a feed-forward network with residual connections, expressed as

$$\begin{aligned} \hat{\mathbf{O}}^{(m)} &= \text{LayerNorm}(\mathbf{x}_d^{(m)} + \mathbf{O}^{(m)}), \\ \mathbf{O}_{\text{time}}^{(m)} &= \text{LayerNorm}(\hat{\mathbf{O}}^{(m)} + \text{MLP}(\hat{\mathbf{O}}^{(m)})), \end{aligned} \quad (3)$$

where MLP denotes a multi-layer (two-layer in this paper) feed-forward network, and $\mathbf{O}_{\text{time}}^{(m)}$ is the output of CTAL.

The computation complexity of the CTAL is $O(MN^2)$. After this stage, temporal dependencies among patches within the feature dimension are encapsulated in $\mathbf{O}_{\text{time}}^{(m)} \in \mathbb{R}^{d_{\text{model}} \times N}$. Subsequently, $\mathbf{O}_{\text{time}} \in \mathbb{R}^{M \times d_{\text{model}} \times N}$ undergoes further processing by CDAL.

Cross-dimension Attention Layer: CDAL needs to apply MSA to all feature dimensions. However, implementing this directly results in a computational complexity of $O(M^2)$, which becomes impractical for a large feature dimension M . To address this problem, we introduce the router mechanism [40]. As depicted in Fig. 2(d), a learnable vector serves as the router for n -th patch, denoted as $\mathbf{R}_n \in \mathbb{R}^{c \times d_{\text{model}}}$, $c \ll M$, $n = 1, \dots, N$. The router aggregates messages from all dimensions using $\mathbf{R}_n$ as query matrices in MSA, and vectors of all dimensions $\mathbf{O}_{\text{time}_n} \in \mathbb{R}^{M \times d_{\text{model}}}$ act as key and value matrices. Subsequently, $\mathbf{R}_n$ redistributes received messages among dimensions by utilizing vectors of dimensions $\mathbf{O}_{\text{time}_n}$ as queries and the aggregated messages as key and value. This establishes an all-to-all connection among M dimensions. The router mechanism can be expressed as:

$$\begin{aligned} \mathbf{A}_n &= \text{MSA}_{\text{dim}}^1(\mathbf{R}_n, \mathbf{O}_{\text{time}_n}, \mathbf{O}_{\text{time}_n}), \quad n = 1, \dots, N. \\ \tilde{\mathbf{O}}_{\text{dim}_n} &= \text{MSA}_{\text{dim}}^2(\mathbf{O}_{\text{time}_n}, \mathbf{A}_n, \mathbf{A}_n), \quad n = 1, \dots, N. \end{aligned} \quad (4)$$

Here, $\mathbf{O}_{\text{time}_n} \in \mathbb{R}^{M \times d_{\text{model}}}$ represents the vectors of all dimensions, $\mathbf{A}_n \in \mathbb{R}^{c \times d_{\text{model}}}$ denotes the aggregated messages from all feature dimensions, and $\tilde{\mathbf{O}}_{\text{dim}_n}$ is the output of the router mechanism. The multi-head attention block also includes LayerNorm layers and a feed-forward network with residual connections, expressed as:

$$\begin{aligned} \hat{\mathbf{O}}_{\text{dim}} &= \text{LayerNorm}(\mathbf{O}_{\text{time}} + \tilde{\mathbf{O}}_{\text{dim}}) \\ \mathbf{O}_{\text{dim}} &= \text{LayerNorm}(\hat{\mathbf{O}}_{\text{dim}} + \text{MLP}(\hat{\mathbf{O}}_{\text{dim}})) \end{aligned} \quad (5)$$

CAM comprises K ($K = 3$ in our work) criss-cross attention blocks (a combination of a CTAL and a CDAL). This process can be expressed as $\mathbf{Z} = \text{CAM}(\{\mathbf{x}_d^{(m)}\}_{m=1}^M)$, where $\mathbf{Z} \in \mathbb{R}^{M \times N \times d_{\text{model}}}$ is the output of CAM. Since the router mechanism reduces the complexity of CDAL from $O(M^2 \sim N)$ to $O(MN)$. The overall computational complexity of CAM is $O(MN^2 + MN) = O(MN^2)$. After CAM, both temporal correlations within the univariate feature series and across multiple feature series are captured in $\mathbf{Z}$.

C. Dual Output Head

With temporal insights of encrypted traffic captured in $\mathbf{Z}$, a dual output head is designed to realize classification and forecasting tasks. The dual output head includes a classification head and a forecasting head, whose architecture is depicted in Fig. 2(e) and (f).

Classification Head: For classification tasks, a straightforward way to obtain label series is passing $\mathbf{Z} \in \mathbb{R}^{M \times N \times d_{\text{model}}}$ through an MLP network to aggregate temporal and cross-dimension information, and then reshaping it to the desired format. However, in the phase of patching and CAM, temporal information is preserved and extracted among patches rather than individual data points, posing challenges for data point classification. Furthermore, applying MLP raises concerns regarding computational complexity. Specifically, the dimension of the input layer being $M \times N \times d_{\text{model}}$ is typically large, leading to elevated learning costs for MLP. To address these challenges, we propose a CNN-based output head for classification tasks. The architecture of the CNN-based head consists of two convolutional blocks, each comprising a convolution layer, an activation layer, and a pooling layer. The convolutional layers are utilized to reconstruct fine-grained temporal information from $\mathbf{Z}$ by capturing features from different channels, while the pooling layers facilitate the extraction of dominant features. After two convolution blocks, the output vectors at the data point level and can be fed into an MLP layer to obtain predicted label series $\hat{\mathbf{y}}$.

Forecasting Head: As illustrated in Fig. 2(f), the forecasting head obtains the forecasting results by pushing $\mathbf{Z}$ through a flattening layer and an MLP. The flattening layer receives the output from the attention module, $\mathbf{Z} \in \mathbb{R}^{M \times N \times d_{\text{model}}}$, and reshapes it into $\tilde{\mathbf{Z}} \in \mathbb{R}^{N \times \Delta}$, where $\Delta = M \times d_{\text{model}}$ represents the flattened dimension of each patch. This process effectively consolidates the information from each patch into a unified vector $\tilde{\mathbf{Z}}_n$, $n = 1, \dots, N$. MLP then processes $\tilde{\mathbf{Z}}$ to extract further insights and produce the forecasted outputs $\hat{\mathbf{y}} = (\hat{y}_{L+1}, \dots, \hat{y}_{L+T})$.

D. Training CTT-Based Classifier/Forecaster

1) *Training CTT-Based Classifier*: As aforementioned, encrypted traffic can be characterized by flow-level or packet-level multivariate feature series, denoted by $\mathcal{X} = \{\mathbf{x}_\tau\}_{\tau=1}^\infty$. Training of CTT-based traffic classifier also requires the ground-truth labels $\mathcal{Y} = \{y_\tau\}_{\tau=1}^\infty$ for the feature series.

Packet/flow-level Traffic Classification: For packet-/flow-level traffic classification, the mini-batch update is used to train CTT. In each training iteration, a mini-batch of feature series $\mathcal{X}_b = \{\vec{\mathbf{x}}_i\}_{i=1}^B$ is randomly sampled from $\mathcal{X}$, where B is the batch size (i.e., number of sampled feature series) and $\vec{\mathbf{x}}_i = \{\mathbf{x}_{i,1}, \mathbf{x}_{i,2}, \dots, \mathbf{x}_{i,L}\}$ is the i -th feature series sample with its starting point $\mathbf{x}_{i,1}$ randomly selected in $\mathcal{X}$. We also obtain the ground-truth label series from $\mathcal{Y}$ for the sampled feature series mini-batch, denoted by $\mathcal{Y}_b = \{\vec{\mathbf{y}}_i\}_{i=1}^B$ where $\vec{\mathbf{y}}_i = \{y_{i,1}, y_{i,2}, \dots, y_{i,L}\}$ is ground-truth label series for $\vec{\mathbf{x}}_i$.

CTT-based encrypted traffic classifier takes the mini-batch of feature series $\mathcal{X}_b$, and generates a mini-batch of predicted label series $\hat{\mathcal{Y}}_b = \{\hat{\vec{\mathbf{y}}}_i\}_{i=1}^B$, where $\hat{\vec{\mathbf{y}}}_i = \{\hat{y}_{i,1}, \hat{y}_{i,2}, \dots, \hat{y}_{i,L}\}$ is the predicted label series for $\vec{\mathbf{x}}_i$. The classification loss is quantified by a cross-entropy loss function $\ell = \sigma(\mathcal{Y}_b, \hat{\mathcal{Y}}_b)$, and parameters of CTT are trained toward minimizing loss.

Packet-to-flow Level Traffic Classification: For p2f-level traffic classification, the mini-batch of packet-level feature series $\mathcal{X}_b = \{\vec{\mathbf{x}}_i\}_{i=1}^B$ and ground-truth label series $\mathcal{Y}_b = \{\vec{\mathbf{y}}_i\}_{i=1}^B$ are constructed. The form of feature series sample $\vec{\mathbf{x}}_i = \{\mathbf{x}_{i,1}, \dots, \mathbf{x}_{i,L}\}$ in p2f-level analysis is identical to the packet-level analysis. The key difference lies in the ground-truth label series which is expressed as $\vec{\mathbf{y}}_i = \{y_{i,j}\}_{j=1}^{L/F}$, where $y_{i,j}$ is the ground-truth label for the j -th flow, $(\mathbf{x}_{i,(j-1) \times F + 1}, \dots, \mathbf{x}_{i,j \times F})$, in $\vec{\mathbf{x}}_i$ and F is the length of the segmented flow. In each training iteration, $\mathcal{X}_b$ is fed into a CTT-based classifier to obtain predicted p2f-level label series $\hat{\mathcal{Y}}_b = \{\hat{\vec{\mathbf{y}}}_i\}_{i=1}^B$. Similarly, a cross-entropy loss function $\ell = \sigma(\mathcal{Y}_b, \hat{\mathcal{Y}}_b)$ is employed for the model training.

2) *Training of CTT-Based Forecaster*: Training of the CTT-based forecaster also utilizes mini-batch updating. In each iteration, we construct a mini-batch of lookback feature series $\mathcal{X}_b = \{\vec{\mathbf{x}}_i\}_{i=1}^B$. A sampled lookback feature series $\vec{\mathbf{x}}_i = \{\mathbf{x}_{i,1}, \dots, \mathbf{x}_{i,L}\}$ has the same form as that in classification tasks. Differing from the CTT-based classifiers, CTT-based forecasters construct mini-batches for two types of ground-truth label series. The first type is the mini-batch for ground-truth lookback label series $\mathcal{Y}_b = \{\vec{\mathbf{y}}_i\}_{i=1}^B$, where $\vec{\mathbf{y}}_i = \{y_{i,1}, \dots, y_{i,L}\}$. The second type is the mini-batch of forthcoming series $\mathcal{Y}_b'' = \{\vec{\mathbf{y}}_i''\}_{i=1}^B$ where $\vec{\mathbf{y}}_i'' = \{y_{i,L+1}, \dots, y_{i,L+T}\}$. The CTT-based forecaster can take two kinds of input: (1) a lookback feature series, $\vec{\mathbf{x}}_i$; (2) a lookback feature series combined with its ground-truth lookback label series, $\{\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i\}$. We have considered both cases in the experiments. Given $\mathcal{X}_b$ or $\{\mathcal{X}_b, \mathcal{Y}_b'\}$, CTT-based forecaster outputs a mini-batch of forecasted label series $\hat{\mathcal{Y}}_b = \{\hat{\vec{\mathbf{y}}}_i\}$, where $\hat{\vec{\mathbf{y}}}_i = \{\hat{y}_{i,L+1}, \dots, \hat{y}_{i,L+T}\}$. The loss function to be minimized by the CTT-based forecaster is also the cross-entropy loss function $\ell = \sigma(\mathcal{Y}_b'', \hat{\mathcal{Y}}_b)$.

V. EXPERIMENTS AND EVALUATIONS

A. Experimental Setups

Datasets and Preparations: We evaluate CTT's performance using 5 real-world datasets: 3 for fingerprinting attacks and 2 for

malware detection. These datasets are selected to enable consistent comparison with SOTA methods, particularly ET-BERT [15] and YaTC [32]. Statistical details of all datasets are provided in Supplementary Material C-A.

1) ISCX-VPN2016 [41] investigates fingerprinting attacks, where adversaries attempt to classify encrypted VPN traffic into 7 user behavior categories (e.g., Browsing, Chat).

2) ISCX-Tor2016 [42] focuses on fingerprinting attacks within Tor-encrypted traffic, covering 8 behavior categories (e.g., Audio, Browsing, Chat).

3) CSTNETTLS1.3 [15] is designed for website fingerprinting attacks over the TLS 1.3 protocol. It comprises traffic traces from 120 websites (e.g., apple.com) under CSTNET.

4) USTC-TFC2016 [43] supports malware detection from a defensive perspective, containing encrypted traffic from 20 software types—10 benign (e.g., BitTorrent, Facetime, FTP) and 10 malicious (e.g., Cridex, Geodo, Htbot).

5) CIC-IoT2022 [44] also targets malware detection. Following [32], we select 10 software categories due to limited traffic in some, comprising seven benign (e.g., Interactions Audio, Interactions Cameras) and three malicious (e.g., Attacks Flood, Attacks Hydra) categories.

We adopt two data processing approaches to evaluate the robustness of CTT under different preprocessing strategies comprehensively. For ISCX-VPN2016, ISCX-Tor2016, USTC-TFC2016, and CIC-IoT2022, we follow a unified pipeline consisting of 3 main steps: (1) constructing multivariate feature series with Tranalyzer2 [37], including session and flow identifiers; (2) mitigating class imbalance through stratified downsampling based on session or flow IDs; and (3) train-test-validation set split with a 70:20:10 ratio, ensuring that each session appears exclusively in one subset to prevent information leakage. For CSTNET-TLS1.3, we adopt a PCAP-level processing strategy, where (1) entire PCAP files serve as the units for the train-test-validation set split with a 70:20:10 ratio. Subsequently, (2) multivariate feature construction and (3) class balancing are performed independently within each subset, following the same procedures as previously described. Detailed preprocessing procedures are provided in Supplementary Material C-B.

Evaluation Metrics and Implementation Details: Four widely recognized evaluation metrics, accuracy (ACC), recall (REC), precision (PRE), and F1-score (F1), are employed to assess the classification and forecasting performance. For CTT implementation, CAM is composed of 3 criss-cross attention blocks. Flow-level classification utilizes a lookback length $L = 64$, balancing essential temporal pattern capture against performance degradation from irrelevant long-range dependencies at higher L values, while preventing insufficient long-term correlation capture at lower L . For p2f analysis, $L = 32$ is empirically selected to avoid excessive short-flow filtering and maintain dataset integrity. Training employed a batch size of 128 and an initial learning rate of 1×10^{-4} , dynamically adjusted via a OneCycle scheduler. To prevent overfitting, early stopping with a patience of 20 epochs is applied. All configurations are evaluated over five independent runs with randomized seeds, with average performance reported. Hyperparameters are summarized in Table II, and features are detailed in Supplementary Material C-C.

B. Observations of “time” Impact

We present several empirical observations in Fig. 3 which are obtained during this work. We characterize real-world encrypted

TABLE II
HYPER-PARAMETERS OF CTT

Variable	Description	Default Value	
		Classify	Forecast
F	Length of flow	32	N/A
P	Patch length	8	4
S	Stride length	4	2
H	Num. of attention heads	16	16
d_{model}	Dimension of latent space	128	128
c	Router dimension	10	10

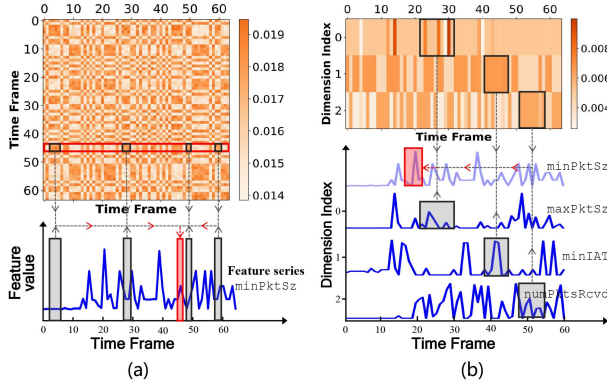


Fig. 3. Illustration of temporal correlations of encrypted traffic. (a) Temporal correlations within univariate feature series. (b) Temporal correlations across feature dimensions.

traffic in dataset ISCX-VPN2016 [41] by extracting features from its flows (minPktSz, maxPktSz, minIAT, numPktsRcvd), and generate a multivariate feature series.

Long-/Short-term temporal correlation in univariate feature series: Fig. 3 (a) presents a visualization of a univariate feature series and its temporal correlations. The bottom part of Fig. 3 (a) depicts the univariate feature series (minPktSz) of encrypted traffic. We compute pairwise correlations of arbitrary time frames within the feature series. The correlation scores, obtained using Scaled Dot-Product Attention [39] (detailed in Supplementary Material A), are presented in a correlation matrix in the upper part of Fig. 3 (a). For a selected target time frame (colored in red), we identify time frames with high correlation scores as its relevant time frames (colored in gray). Notably, relevant time frames exhibit varying distances from the target time frame, some distant and others being proximate. This observation underscores the presence of both long-term and short-term temporal correlations within encrypted traffic.

Temporal correlation across feature dimensions: Fig. 3 (b) presents visualizations of multiple univariate feature series and the temporal correlations across different feature dimensions. The bottom part of Fig. 3 (b) depicts a multivariate feature series of the encrypted traffic. Within this context, we select a specific target frame in the minPktSz feature series and assess its temporal correlations with time frames in other feature dimensions. The upper part of Fig. 3 (b) depicts the correlation score, which demonstrates noticeable temporal correlations between time frames across multiple feature dimensions.

C. Comparison With State-of-The-Art Methods

1) *Traffic Classification. Baselines:* We evaluate CTT against 11 state-of-the-art (SOTA) methods, categorized as

TABLE III
FLOW-LEVEL CLASSIFICATION PERFORMANCE

Dataset	Method	ACC	PRE	REC	F1
ISCX-VPN2016	CTT	0.9810	0.9578	0.9394	0.9481
	FlowPrint [3]	0.9473	0.8868	0.8799	0.8692
	AppScanner [10]	0.5534	0.7480	0.2950	0.3504
	Deeppacket [34]	0.7572	0.6640	0.6797	0.6300
	ET-BERT [15]	0.8698	0.8894	0.8695	0.8765
USTC-TFC2016	CTT	0.9952	0.9913	0.9910	0.9912
	FlowPrint [3]	0.9423	0.9395	0.9228	0.9282
	AppScanner [10]	0.9334	0.9523	0.8619	0.9024
	Deeppacket [34]	0.9210	0.9075	0.8905	0.8923
	ET-BERT [15]	0.9342	0.9634	0.9436	0.9384
CIC-IoT2022	CTT	0.9967	0.9365	0.8424	0.8829
	FlowPrint [3]	0.9982	0.6756	0.7271	0.6792
	AppScanner [10]	0.9758	0.8182	0.4594	0.5320
	Deeppacket [34]	0.9804	0.4684	0.3621	0.3648
	ET-BERT [15]	0.8636	0.7667	0.7881	0.7640
CSTNET-TLS1.3	CTT	0.9399	0.9314	0.9210	0.9248
	FlowPrint [3]	0.3720	0.3765	0.3186	0.2905
	AppScanner [10]	0.6311	0.6226	0.5829	0.5672
	Deeppacket [34]	0.5499	0.5350	0.4967	0.4701
	ET-BERT [15]	0.9465	0.9455	0.9288	0.9322

¹ Rows highlighted in gray represent the proposed method.

² Bold values indicate the best performance across all methods.

follows: (1) ML-based methods: FlowPrint [3] and AppScanner [10]; (2) CNN/RNN-based methods: Deep Fingerprinting (DF) [27], FS-Net [14], Deeppacket [34], and TSCRNN [28]; (3) Transformer-based methods: ET-BERT [15], YaTC [32] and AN-Net [33]; (4) Multimodal methods: App-Net [45] and MIMETIC [46]. As established in [45] and [46], both App-Net and MIMETIC require two input modalities: traffic-feature-based time series and traffic payload. Since our datasets lack payload information, these models cannot be directly applied. To enable fair comparison, we introduce modified versions, AppNet-LSTM and MIMETIC-GRU, retaining only their RNN and GRU components respectively to capture temporal correlations. For further details on baselines, refer to Supplementary Material C-D. Additionally, as most SOTA methods are designed for specific analytical levels in traffic classification (see Table I), each baseline is compared only at its applicable analytical level.

Results and Discussions: The comparative analysis of CTT and baselines at flow-level is summarized in Table III. CTT achieves superior performance in encrypted traffic classification across 3 datasets, with F1-scores of 0.9481, 0.9912, and 0.8829. Flow-level classification on ISCX-Tor2016 is omitted due to its limited size (469 flows) and severe class imbalance. On CSTNET-TLS1.3, the scarcity of per-class samples, combined with ET-BERT's stronger small-sample learning capability as a pre-training-based method, results in slightly better performance than CTT. The packet-level classification performance is presented in Table IV. CTT consistently achieves values exceeding 0.91 across all evaluation metrics over all 5 datasets, underscoring its robust capability in capturing temporal dependencies among packet sequences.

Classification performance at the p2f-level is detailed in Table V. Evaluation of the p2f-level in CSTNet-TLS1.3 is precluded due to severe class imbalance. CTT outperforms other methods on ISCX-VPN2016, ISCX-Tor2016, and USTC-TFC2016 datasets, achieving F1-scores of 0.9943, 0.9962, and 0.9948, respectively. However, CTT encounters challenges on the CIC-IoT2022 dataset due to significant category imbalance, resulting in comparatively lower performance on minority classes. Conversely, YaTC exhibits remarkable effectiveness in

TABLE IV
PACKET-LEVEL CLASSIFICATION PERFORMANCE

Dataset	Method	ACC	PRE	REC	F1
ISCX-VPN2016	CTT	0.9998	0.9992	0.9992	0.9992
	AppScanner [10]	0.8398	0.7500	0.5931	0.6533
	Deeppacket [34]	0.9858	0.9329	0.9826	0.9526
	ET-BERT [15]	0.9860	0.9858	0.9861	0.9858
ISCX-Tor2016	CTT	0.9983	0.9979	0.9962	0.9970
	AppScanner [10]	0.9890	0.8889	0.8629	0.8752
	Deeppacket [34]	0.9721	0.9641	0.9123	0.9298
	ET-BERT [15]	0.9843	0.9841	0.9843	0.984
USTC-TFC2016	CTT	0.9952	0.995	0.9950	0.9950
	AppScanner [10]	0.9159	0.9519	0.8181	0.8725
	Deeppacket [34]	0.9545	0.8666	0.8491	0.8544
	ET-BERT [15]	0.9635	0.9728	0.9652	0.9652
CIC-IoT2022	CTT	0.9958	0.9178	0.9106	0.9127
	AppScanner [10]	0.9838	0.9091	0.6777	0.7349
	Deeppacket [34]	0.9718	0.7513	0.6235	0.6538
	ET-BERT [15]	0.9842	0.7990	0.7792	0.8260
CSTNET-TLS1.3	CTT	0.9764	0.9631	0.9684	0.9648
	AppScanner [10]	0.4798	0.5844	0.6118	0.5704
	Deeppacket [34]	0.4251	0.5507	0.5420	0.5127
	ET-BERT [15]	0.9647	0.9563	0.9567	0.9557

TABLE V
P2F-LEVEL CLASSIFICATION PERFORMANCE

Dataset	Method	ACC	PRE	REC	F1
ISCX-VPN2016	CTT	0.9987	0.9935	0.9950	0.9943
	DF [27]	0.6886	0.2102	0.2800	0.2358
	FS-Net [14]	0.8949	0.8983	0.8590	0.8700
	TSCRNN [28]	0.9769	0.9251	0.9700	0.9394
	YaTC [32]	0.9650	0.9601	0.9370	0.9475
	AN-Net [33]	0.9899	0.9612	0.9603	0.9607
	AppNet-LSTM [45]	0.9805	0.8777	0.9048	0.8870
	MIMETIC-GRU [46]	0.9825	0.8830	0.9073	0.8913
ISCX-Tor2016	CTT	0.9980	0.9969	0.9955	0.9962
	DF [27]	0.5603	0.2473	0.3444	0.2761
	FS-Net [14]	0.6024	0.4018	0.4608	0.4233
	TSCRNN [28]	0.9327	0.7965	0.8001	0.7964
	YaTC [32]	0.9863	0.9771	0.9881	0.9818
	AN-Net [33]	0.9847	0.9781	0.9342	0.9509
	AppNet-LSTM [45]	0.9151	0.8706	0.7793	0.7856
	MIMETIC-GRU [46]	0.9248	0.9088	0.8285	0.8467
USTC-TFC2016	CTT	0.9961	0.9947	0.9950	0.9948
	DF [27]	0.5337	0.2337	0.2684	0.2340
	FS-Net [14]	0.8705	0.8761	0.8432	0.8384
	TSCRNN [28]	0.9887	0.9714	0.9707	0.9708
	YaTC [32]	0.9684	0.9761	0.9761	0.9761
	AN-Net [33]	0.9854	0.9629	0.9488	0.9519
	AppNet-LSTM [45]	0.9813	0.9813	0.9187	0.9260
	MIMETIC-GRU [46]	0.9885	0.9614	0.9612	0.9603
CIC-IoT2022	CTT	0.9949	0.9241	0.9237	0.9198
	DF [27]	0.9741	0.5630	0.5626	0.5617
	FS-Net [14]	0.9704	0.7346	0.6076	0.6320
	TSCRNN [28]	0.9877	0.7795	0.7462	0.7501
	YaTC [32]	0.9532	0.9528	0.9309	0.9394
	AN-Net [33]	0.9855	0.8581	0.8243	0.8264
	AppNet-LSTM [45]	0.9842	0.8031	0.6997	0.7343
	MIMETIC-GRU [46]	0.9856	0.7831	0.7631	0.7666

this scenario, benefiting from its robust transfer learning capabilities. The performance gaps observed in specific scenarios stem from the divergence between CTT’s “training-from-scratch” approach and the “pre-training” paradigms of baselines like ET-BERT and YaTC. Although pre-training improves robustness in data-scarce environments via prior knowledge, it necessitates significant computational resources.

Mitigation of Class Imbalance: To enhance CTT’s robustness against data imbalance in practical deployments, we evaluated two specific mitigation strategies on the highly imbalanced CIC-IoT2022 dataset (p2f-level): (1) Random Oversampling, which

TABLE VI
PERFORMANCE COMPARISON OF IMBALANCE MITIGATION STRATEGIES

Method	ACC	PRE	REC	F1
CTT (Origin)	0.9949	0.9241	0.9237	0.9198
FocalLoss	0.9971	0.9732	0.9439	0.9561
Oversample	0.9816	0.9816	0.9816	0.9816
YaTC [32]	0.9532	0.9528	0.9309	0.9394

TABLE VII
EFFICIENCY COMPARISON ON ISCX-VPN2016 DATASET

Method	F1	PT(ms/iter)	MF(GB)	CPU(%)
CTT	0.9943	91.01	6.66	257.0
DF [27]	0.2358	8.90	1.75	57.4
FS-Net [14]	0.8700	46.50	2.05	86.5
TSCRNN [28]	0.9394	47.10	2.47	63.2
YaTC [32]	0.9475	108.20	3.75	100.3

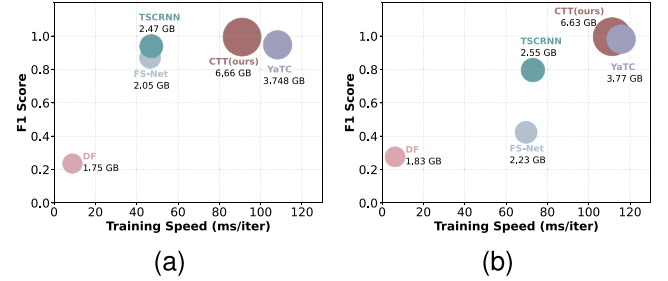


Fig. 4. Efficiency comparison at the p2f-level. Circle size is proportional to memory footprint. (a) VPN2016. (b) Tor2016.

augments minority classes to the median sample frequency using random oversampling with replacement; and (2) FocalLoss [47], configured with $\alpha = 0.25$ and $\gamma = 2$ to focus learning on hard examples. The comparative results, summarized in Table VI, demonstrate that both strategies yield distinct performance gains compared to the baseline YaTC. These findings confirm that integrating sampling-based augmentation or specialized loss functions effectively improves CTT’s capability to handle imbalanced distributions, offering actionable solutions for real-world network environments.

Efficiency Analysis: We conduct a comprehensive comparison of classification performance, processing time (PT), memory footprint (MF), and CPU utilization between CTT and baseline models at the packet-to-flow (p2f) level. Processing time is measured using training speed per iteration. The results are summarized in Table VII, with a visual representation provided in Fig. 4. As shown in Table VII, CTT achieves a faster processing speed than other Transformer-based methods (YaTC) but remains slower than other deep learning-based approaches. Meanwhile, Transformer-based models (CTT and YaTC) exhibit superior classification performance compared to other baselines but entail a higher memory footprint and increased CPU utilization during training.

Significance Tests: Statistical significance tests are conducted on the ISCX-VPN2016 and ISCX-Tor2017 datasets to evaluate the performance of CTT in comparison to baseline methods at the packet-level. The evaluations on each dataset are performed over 10 independent runs with different random seeds. In addition to the 5 runs presented in the main results, 5 additional runs are conducted to enhance the robustness of the statistical analysis. Wilcoxon signed-rank tests are employed to compare CTT with each baseline, consistently demonstrating that CTT significantly outperforms all baseline methods ($W = 55.0$,

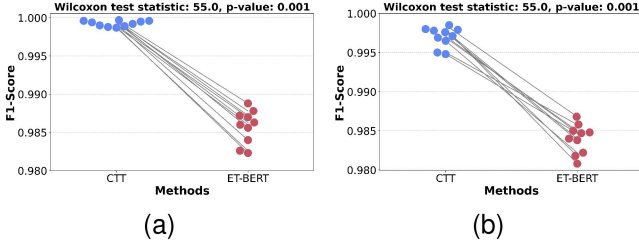


Fig. 5. Paired performance plots of CTT and ET-BERT at packet-level. (a) ISCX-VPN2016. (b) ISCX-Tor2016.

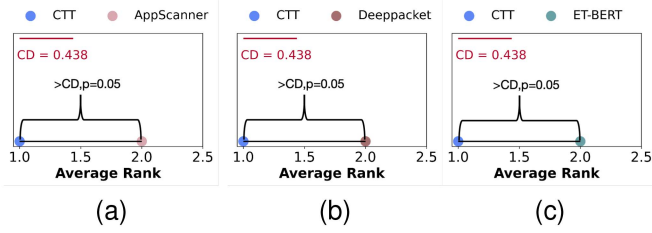


Fig. 6. Critical Difference (CD) diagram at packet-level. (a) AppScanner. (b) Deeppacket. (c) ET-BERT.

TABLE VIII
CLASSIFICATION PERFORMANCE UNDER DATA SCARCITY

Ratio	ACC(Gap)	PRE(Gap)	REC(Gap)	F1(Gap)
1%	0.922 (↓7.58%)	0.893 (↓10.41%)	0.854 (↓14.20%)	0.869 (↓12.74%)
2%	0.949 (↓4.88%)	0.892 (↓10.54%)	0.895 (↓10.05%)	0.893 (↓10.38%)
5%	0.990 (↓0.84%)	0.973 (↓2.41%)	0.949 (↓4.71%)	0.959 (↓3.73%)
10%	0.992 (↓0.6%)	0.970 (↓2.68%)	0.965 (↓3.07%)	0.968 (↓2.88%)
100%	0.998	0.997	0.996	0.996

$p = 0.001$) across both datasets. Paired performance plots, presented in Fig. 5, illustrate that CTT consistently outperforms the previous SOTA, ET-BERT, across varying random seeds. Furthermore, the Nemenyi test is employed, facilitating pairwise comparisons across multiple datasets. The calculated Critical Difference (CD) is 0.438 ($p = 0.05$). CD diagrams illustrating the performance of CTT relative to each baseline on the two datasets are presented in Fig. 6. The ranking differences between CTT and each baseline exceed the CD threshold, thereby confirming that CTT demonstrates statistically significant superiority over all baseline methods.

Data Scalability Analysis: To assess CTT's robustness under limited labeled data, we conducted additional experiments on ISCX-Tor2017 for packet-to-flow level traffic classification. We trained CTT using only 1%, 2%, 5%, and 10% of the original training data while keeping the validation and test sets unchanged, ensuring temporal continuity in the remaining training sequences. The results, presented in Table VIII, show that with 1% or 2% of labeled data, CTT's F1-score declines by 12.74%. However, with 5% or 10%, performance remains stable, with a reduction of less than 5%. These findings underscore CTT's robustness, demonstrating its ability to maintain competitive performance under data scarcity.

2) Traffic Forecasting: Baselines: In encrypted traffic forecasting, there appears to be a paucity of related works, to the best of our knowledge. To evaluate the effectiveness of CTT, we benchmark it against two SOTA time-series Transformers: PatchTST [18] and Crossformer [40]. Given the typically short

TABLE IX
FORECASTING PERFORMANCE OF CTT AND BASELINES

Forecast Length	Dataset Methods	ISCX-VPN2016		USTC-TFC2016	
		ACC	F1	ACC	F1
8	CTT	0.9601	0.7407	0.9295	0.8880
	PatchTST [18]	0.9922	0.6859	0.9247	0.8810
	Crossformer [40]	0.9326	0.6680	0.9250	0.8895
12	CTT	0.9488	0.7101	0.9269	0.8818
	PatchTST [18]	0.9072	0.5782	0.9241	0.8778
	Crossformer [40]	0.9220	0.6167	0.9254	0.8805
24	CTT	0.9335	0.6695	0.9252	0.8786
	PatchTST [18]	0.9111	0.5877	0.9217	0.8737
	Crossformer [40]	0.9326	0.6305	0.9233	0.8736

TABLE X
EFFICIENCY COMPARISON ON FORECASTING TASKS

Method	F1	PT(ms/iter)	MF(GB)	CPU(%)
CTT	0.6695	95.40	3.7	556.6
PatchTST [18]	0.5877	85.50	2.5	884.8
Crossformer [40]	0.6305	98.80	3.8	1039.0

packet inter-arrival times, forthcoming packets may arrive before the forecasted results can be effectively utilized. Therefore, this experiment focuses on forecasting flow-level label sequences with a lookback length of $L = 24$. The experiments are conducted with varying forecasting steps ($T = 8, 12, 24$) on the ISCX-VPN2016 and USTC-TFC2016 datasets. The selection of T is driven by the inherently bursty and non-stationary nature of encrypted network traffic, which imposes fundamental constraints on reliably predicting label sequences over extended horizons. Consequently, our evaluation prioritizes short-term traffic forecasting. Additionally, the limited number of flows in ISCX-Tor2016 and the imbalanced class distribution in CIC-IoT2022 pose significant challenges for forecasting tasks, leading to their exclusion from this part.

Results and Discussions: Table IX presents the forecasting performance of CTT in comparison to baselines. Specifically, on the ISCX-VPN2016 dataset, CTT achieves F1-scores of 0.7407, 0.7101, and 0.6695 at different forecasting steps, demonstrating a notable improvement over baselines. On the USTC-TFC2016 dataset, CTT shows comparable performance to baselines, with F1-scores of 0.888, 0.8818, and 0.8786 at evaluated forecasting steps. These results validate the efficacy of CTT on traffic forecasting tasks.

Efficiency Analysis: We further compare the efficiency of CTT with baselines for the traffic forecasting task. The results on the ISCX-VPN2016 dataset with $T = 24$ are summarized in Table X. While PatchTST achieves the best processing speed and memory efficiency, CTT outperforms the baselines in terms of CPU utilization and forecasting performance.

Performance without Lookback Label: We also assess the forecasting performance of CTT under conditions where the lookback label series is unavailable. The F1-scores of CTT and baselines, with and without access to lookback label series on ISCX-VPN2016 and USTC-TFC2016 datasets, are illustrated in Fig. 7. As illustrated, the absence of ground-truth labels from the lookback traffic series does not significantly impair the performance of CTT. Notably, CTT maintains superior forecasting accuracy without lookback traffic labels, achieving F1-scores of 0.7361, 0.67, and 0.5955 for different forecasting lengths on ISCX-VPN2016, and 0.8831, 0.8739, and 0.8768 on USTC-TFC2016.

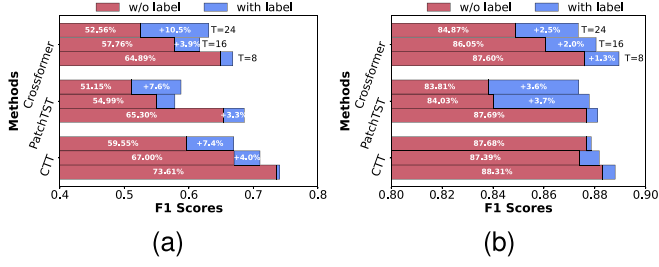


Fig. 7. Forecasting performance (F1 score) w/o. lookback labels under varying forecasting lengths T . (a) ISCX-VPN2016. (b) USTC-TFC2016.

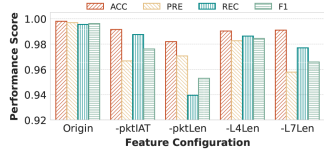


Fig. 8. Impact of individual feature omission.

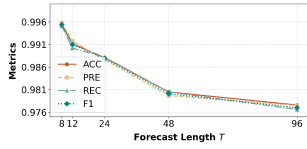


Fig. 9. Open-world binary forecasting performance.

Open-Set Forecasting Capability: To assess CTT’s robustness in dynamic, open-world environments, we conducted an open-set binary forecasting experiment on the USTC-TFC2016 dataset under a label-free setting. We established an open-set protocol by training on a subset of 16 classes while reserving four categories—Facetime and Skype (Benign), alongside Htbot and Nsis-ay (Malicious)—exclusively for the test set as unseen classes. With a fixed lookback length of $L = 24$, the performance across varying forecast horizons (T) is illustrated in Fig. 9. Notably, CTT exhibits strong generalization capabilities, maintaining an F1-score of 0.977 even at an extended horizon of $T = 96$. These results validate CTT’s effectiveness in distinguishing novel malicious threats from benign traffic without prior exposure.

D. Ablation Studies

1) Component Ablation: Recall that CTT encompasses three key components: Channel-Independent Patching (CIP) mechanism, Criss-cross Attention Module (CAM), and Dual Output Heads (DOH). We conduct ablation studies on these components for the encrypted traffic classification task.

The experimental results for the traffic classification task are summarized in Table XI, with ET-BERT [15] serving as a baseline. In CTT without CI, channel-mixing patching replaces the channel-independent paradigm. This involves projecting each univariate feature series $\mathbf{x}_t$ into a latent space, $\mathbf{x}_t \rightarrow \mathbf{h}_t$, $\mathbf{h}_t \in \mathbb{R}^{d_{\text{mix}}}$, where $d_{\text{mix}} = 10$ (equal to c), preventing independent extraction of temporal correlations within each feature. Additionally, CTT without CI excludes CDAL in the attention module, meaning temporal correlations across multiple

features are implicitly captured through channel-mixing rather than explicitly via CDAL. CTT without CDAL retains only CTAL in the attention module, omitting CDAL. CTT without DOH replaces the CNN-based output head for traffic classification with an MLP head of equivalent depth.

As illustrated in Table XI: (1) The absence of the channel-independent mechanism leads to performance degradation across all configurations, highlighting the crucial role of capturing temporal correlations within individual feature series. (2) Removing CDAL from CTT results in inferior performance in most configurations, emphasizing the importance of capturing temporal correlations across multiple features. (3) Replacing the CNN head with an MLP head in CTT configurations consistently results in performance degradation, indicating the superior capability of the CNN-based head to extract and utilize feature information effectively. (4) The integration of CIP, CAM, and DOH in CTT consistently yields superior classification results across most settings.

The effect of the patching strategy is presented in Supplementary Material D-B.

2) Impact of Specific Features: To elucidate the methodological source of CTT’s robustness, we conducted a feature study on the ISCX-Tor2016 dataset for the p2f level classification task. We systematically excluded key packet-level features—specifically `pktIAT`, `pktLen`, `L7Len`, and `L4Len`—from the input to evaluate their individual contributions. The resulting performance variations are illustrated in Fig. 8. As observed, the omission of `pktLen` induces the most substantial decline in accuracy, highlighting its critical role in characterizing encrypted traffic. Nevertheless, CTT demonstrates remarkable resilience; despite varying impacts from different features, the model maintains robust performance even when individual key attributes are omitted.

3) Impact of Hyperparameters. Router Dimensions: This experiment studies the impact of router dimension c on the classification performance. We fix the lookback length L to be 64 (32) at the flow/packet (packet-to-flow) level and vary the router dimension to be $c = [5, 10, 20]$. The F1-scores under varying router dimensions are depicted in Fig. 10. As shown, F1-scores remain relatively stable in most settings with varying c , indicating that CTT exhibits insensitivity to changes in c within a certain range.

Stride Lengths: This experiment examines the effects of the stride length S . The traffic classification performance of CTT with varying stride length ($S = [1, 2, 3, 4]$) while the patch length P is fixed to 4 on USTC-TFC2016 and CIC-IoT2022 is summarized in Table XII. We can see that on the USTC-TFC2016 dataset, the performance of CTT does not vary significantly with varying stride lengths. Otherwise, the performance of CTT degrades slightly when the stride length extends. It can be concluded that the performance of CTT with varying S is relatively robust, and the effect can be varied under different network conditions. Further empirical analysis regarding the impact of stride length S and patch length P is provided in Supplementary Material D. Collectively, the results suggest that highly bursty traffic necessitates smaller P values to preserve local resolution, while traffic with rigid heartbeat patterns demonstrates distinct insensitivity to variations in patch size.

Lookback Lengths: We investigate the influence of lookback length L and forecasting length T on prediction performance. As shown in Fig. 11, the optimal lookback length depends on the forecast horizon: shorter L values are effective for

TABLE XI
ABLATION STUDY OF CTT FOR ENCRYPTED TRAFFIC CLASSIFICATION

Dataset	Method	Flow level				Packet level				Packet-to-flow level			
		ACC	PRE	REC	F1	ACC	PRE	REC	F1	ACC	PRE	REC	F1
ISCX-VPN2016	ET-BERT [15]	0.8698	0.8894	0.8695	0.8765	0.9860	0.9858	0.9861	0.9858	N/A	N/A	N/A	N/A
	CTT w.o. CI	0.9321	0.7161	0.6485	0.6676	0.9986	0.9942	0.9929	0.9935	0.9960	0.9899	0.9789	0.9841
	CTT w.o. CDAL	0.9618	0.8841	0.8821	0.8816	0.9995	0.9980	0.9989	0.9985	0.9974	0.9824	0.9963	0.9891
	CTT w.o. DOH	0.9528	0.8593	0.8682	0.8541	0.9992	0.9955	0.9985	0.9970	0.9984	0.9905	0.9954	0.9929
	CTT	0.9810	0.9578	0.9394	0.9481	0.9998	0.9992	0.9992	0.9992	0.9987	0.9935	0.9950	0.9943
ISCX-Tor2016	ET-BERT [15]	N/A				0.9843	0.9841	0.9843	0.9840	N/A	N/A	N/A	N/A
	CTT w.o. CI	(Due to the limited number of flows, flow-level analysis is not performed on the ISCX-Tor2016 dataset.)				0.9892	0.9799	0.9742	0.9770	0.9843	0.9710	0.9523	0.9602
	CTT w.o. CDAL					0.9728	0.9691	0.9489	0.9579	0.9920	0.9879	0.9790	0.9832
	CTT w.o. DOH					0.9901	0.9873	0.9751	0.9808	0.9967	0.9942	0.9848	0.9893
	CTT					0.9983	0.9979	0.9962	0.9970	0.9980	0.9969	0.9955	0.9962
USTC-TFC2016	ET-BERT [15]	0.9342	0.9634	0.9436	0.9384	0.9635	0.9728	0.9652	0.9652	N/A	N/A	N/A	N/A
	CTT w.o. CI	0.9940	0.9891	0.9887	0.9888	0.9862	0.9740	0.9877	0.9758	0.9889	0.9723	0.9493	0.9552
	CTT w.o. CDAL	0.9900	0.9885	0.9879	0.9881	0.9938	0.9902	0.9904	0.9902	0.9953	0.9895	0.9893	0.9892
	CTT w.o. DOH	0.9934	0.9883	0.9865	0.9874	0.9944	0.9922	0.9923	0.9922	0.9958	0.9898	0.9909	0.9903
	CTT	0.9952	0.9913	0.9910	0.9912	0.9952	0.9950	0.9950	0.9950	0.9961	0.9947	0.9950	0.9948
CIC-IoT2022	ET-BERT [15]	0.8636	0.7667	0.7881	0.7640	0.9842	0.7990	0.7792	0.8260	N/A	N/A	N/A	N/A
	CTT w.o. CI	0.9941	0.8682	0.7564	0.7819	0.9909	0.9053	0.8639	0.8800	0.9928	0.8543	0.8335	0.8399
	CTT w.o. CDAL	0.9948	0.8252	0.7752	0.7900	0.9939	0.9012	0.8951	0.8979	0.9950	0.8961	0.9000	0.8965
	CTT w.o. DOH	0.9957	0.8783	0.7874	0.8235	0.9935	0.9034	0.9076	0.9051	0.9943	0.8889	0.8740	0.8723
	CTT	0.9967	0.9365	0.8424	0.8829	0.9958	0.9178	0.9106	0.9127	0.9956	0.9192	0.9188	0.9175

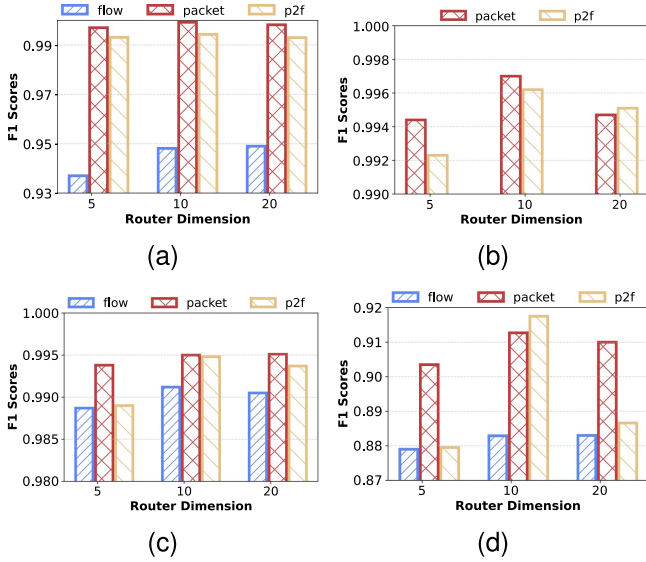


Fig. 10. Classification performance (F1-score) of CTT with different router dimensions. (a) VPN2016. (b) Tor2016. (c) TFC2016. (d) IoT2022.

TABLE XII
CLASSIFICATION PERFORMANCE WITH VARYING S

S	USTC-TFC2016				CIC-IoT2022			
	ACC	PRE	REC	F1	ACC	PRE	REC	F1
1	0.9967	0.9962	0.9963	0.9962	0.9978	0.9268	0.9162	0.9206
2	0.9961	0.9947	0.9950	0.9948	0.9949	0.9241	0.9237	0.9198
3	0.9965	0.9953	0.9952	0.9952	0.9969	0.9232	0.8870	0.8923
4	0.9964	0.9958	0.9962	0.9960	0.9956	0.8924	0.8753	0.8698

short-term forecasting, while medium lengths offer more balanced performance across varying T . On the USTC-TFC2016 dataset, longer lookbacks yield certain advantages, whereas no such benefits are observed on ISCX-VPN2016. This indicates that optimal lookback length is shaped not only by the forecast horizon but also by intrinsic temporal characteristics of the traffic, such as autocorrelation, periodicity, and burstiness. It is observed that predictive performance naturally degrades as the forecasting horizon extends. This phenomenon is primarily driven by the inherent burstiness of encrypted traffic, which

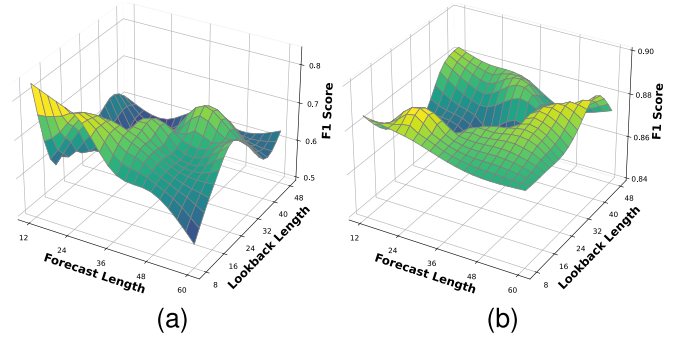


Fig. 11. Forecasting performance (F1-score) of CTT with varying lookback length L and forecasting length T . (a) ISCX-VPN2016. (b) USTC-TFC2016.

leads to the accumulation of uncertainty over longer time steps, posing challenges for long-term dependency modeling. An analysis of performance and resource utilization under varying L is provided in Supplementary Material D-D. Practically, the determination of L should integrate the temporal characteristics of the target traffic with prevailing resource limitations, refined via empirical tuning. In the absence of strong prior knowledge, a moderate L typically serves as a robust default.

To facilitate the efficient identification of optimal configurations for diverse scenarios, we have integrated a dedicated automated grid search module for these hyperparameters into our open-source code implementation¹.

VI. CONCLUSIONS AND FUTURE WORKS

This paper introduced CTT, a novel time series transformer model for encrypted network traffic analysis. The generalizability of CTT is demonstrated through its ability to analyze encrypted traffic at multiple granularities, which renders CTT particularly valuable in web service security contexts.

Limitations and Future Works: The primary limitations of CTT include challenges in long-term traffic forecasting, performance sensitivity in sparse data scenarios, and the reliance

¹https://github.com/Amanda-HuaDing/Criss-cross_Traffic_Transformer.

on a closed-set assumption. To mitigate data scarcity, we plan to incorporate active learning strategies alongside semi-supervised paradigms. This involves developing mechanisms to intelligently identify and annotate the most informative unlabeled samples, thereby maximizing model performance while minimizing annotation costs. To alleviate performance degradation in long-term forecasting scenarios, we aim to investigate ensemble learning techniques and generative approaches with future-sequence attention, effectively mitigating error accumulation over extended horizons. Simultaneously, to overcome the closed-set constraint where unseen classes are inevitably mapped to known representations, we aim to evolve towards open-world scenarios. This involves integrating Open-Set Recognition and anomaly detection mechanisms to effectively distinguish novel threats from established traffic patterns. Regarding efficiency and scalability, future works include incorporating lightweight techniques—such as model pruning and quantization—to enhance efficiency for edge deployment. We also aim to integrate dynamic routing into the Cross-Dimension Attention Layer to adaptively handle high-dimensional data. Furthermore, we identify adaptive configuration as a critical research direction. Moving beyond our current reliance on empirical grid search, we aim to develop environment-aware strategies leveraging automated frameworks such as Bayesian optimization.

Application Scenarios: Beyond encrypted traffic analysis, CTT can be applied to real-world scenarios such as intrusion detection, anomaly detection, and threat intelligence analysis, where identifying subtle temporal patterns in network activity is crucial. Moreover, its forecasting capabilities can support predictive cybersecurity defenses, enabling proactive defense against threats such as encrypted botnet communications or web application abuse by anticipating anomalous encrypted traffic trends. Adapting and practically implementing CTT to diverse implementation scenarios within network security holds significant potential and warrants further exploration.

REFERENCES

- [1] Electronic Frontier Foundation, "Year in review: The last mile of encrypting the web," 2023. [Online]. Available: <https://www.eff.org/deeplinks/2023/12/year-review-last-mile-encrypting-web>
- [2] Zscaler Security Research Team, "2022 encrypted attacks report," 2022. [Online]. Available: [2022EncryptedAttacksReport](https://www.zscaler.com/resources/articles/2022-encrypted-attacks-report)
- [3] T. Van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. System Secur. Symp.*, Feb. 2020, vol. 27.
- [4] C. Fu, Q. Li, K. Xu, and J. Wu, "Point cloud analysis for ML-based malicious traffic detection: Reducing majorities of false positive alarms," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 1005–1019.
- [5] K. Wolsing, E. Wagner, and M. Henze, "Facilitating protocol-independent industrial intrusion detection systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 2105–2107.
- [6] S. Zhu et al., "Measuring and modeling the label dynamics of online {Anti-Malware} engines," in *Proc. USENIX Secur.*, 2020, pp. 2361–2378.
- [7] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, Firstquarter 2023.
- [8] M. Roughan, S. Sen, O. Spatscheck, and N. Duffield, "Class-of-service mapping for QoS: A statistical signature-based approach to IP traffic classification," in *Proc. 4th ACM SIGCOMM Conf. Internet Meas.*, 2004, pp. 135–148.
- [9] D. Bonfiglio, M. Mellia, M. Meo, D. Rossi, and P. Tofanelli, "Revealing Skype traffic: When randomness plays with you," in *Proc. Conf. Appl. Technol., Architectures, Protoc. Comput. Commun.*, 2007, pp. 37–48.
- [10] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy*, May 2016, pp. 439–454.
- [11] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 1723–1732.
- [12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, vol. 25.
- [13] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," 2014, *arXiv:1409.2329*.
- [14] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, Jun. 2019, pp. 1171–1179.
- [15] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [16] A. Graves and A. Graves, "Long short-term memory," in *Supervised Sequence Labelling With Recurrent Neural Networks*. Berlin, Germany: Springer, Jan. 2012, pp. 37–45.
- [17] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDFormer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 27268–27286.
- [18] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. 11th Int. Conf. Learn. Representations*, 2023.
- [19] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2021.
- [20] A. H. Lashkari, A. F. A. Kadir, H. Gonzalez, K. F. Mbah, and A. A. Ghorbani, "Towards a network-based framework for android malware detection and characterization," in *Proc. 15th Annu. Conf. Privacy, Secur. Trust*, Aug. 2017, pp. 233–23309.
- [21] M. Laštovička, S. Špaček, P. Velan, and P. Čeleda, "Using TLS fingerprints for OS identification in encrypted traffic," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2020, pp. 1–6.
- [22] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.
- [23] M. Priymak and R. Sinnott, "Real-time traffic classification through deep learning," in *Proc. IEEE/ACM 8th Int. Conf. Big Data Comput., Appl. Technol.*, 2021, pp. 128–133.
- [24] Y. Huo, Y. Yan, D. Du, Z. Wang, Y. Zhang, and Y. Yang, "Long-term span traffic prediction model based on STL decomposition and LSTM," in *Proc. 20th Asia-Pacific Netw. Operations Manage. Symp.*, Nov. 2019, pp. 1–4.
- [25] Y. Fang, S. Ergüt, and P. Patras, "SDGNet: A handover-aware spatiotemporal graph neural network for mobile traffic forecasting," *IEEE Commun. Lett.*, vol. 26, no. 3, pp. 582–586, Mar. 2022.
- [26] A. Montieri, G. Bovenzi, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapè, "Packet-level prediction of mobile-app traffic using multitask deep learning," *Comput. Netw.*, vol. 200, Dec. 2021, Art. no. 108529.
- [27] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1928–1943.
- [28] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.
- [29] U. Khandelwal, H. He, P. Qi, and D. Jurafsky, "Sharp nearby, fuzzy far away: How neural language models use context," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1, Long Papers)*, 2018, pp. 284–294.
- [30] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 9640–9649.
- [31] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol., Vol. 1 (Long Short Papers)*, 2019, pp. 4171–4186. [Online]. Available: <https://api.semanticscholar.org/CorpusID>
- [32] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.
- [33] X. Deng, Y. Wang, and Z. Xue, "AN-Net: An anti-noise network for anonymous traffic classification," in *Proc. ACM Web Conf.*, 2024, pp. 4417–4428.
- [34] M. Lotfollahi, M. Jafari, R. Siavoshani, S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.

- [35] TCPdump Team, "TCPdump network debugger," (n.d.). [Online]. Available: <https://www.tcpdump.org/>
- [36] Wireshark Foundation, "Wireshark," (n.d.). [Online]. Available: <https://www.wireshark.org/>
- [37] Anteater, "Tranalyzer," (n.d.). [Online]. Available: <https://tranalyzer.com/>
- [38] C. Zhang, M. Fiore, and P. Patras, "Multi-service mobile traffic forecasting via convolutional long short-term memories," in *Proc. IEEE Int. Symp. Meas. Netw.*, 2019, pp. 1–6.
- [39] A. Vaswani et al., "Attention is all you need," *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30.
- [40] Y. Zhang and J. Yan, "CrossFormer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. 11th Int. Conf. Learn. Representations.*, 2023.
- [41] G. D. Gil, A. H. Lashkari, M. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
- [42] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2017, vol. 2, pp. 253–262.
- [43] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw.*, 2017, pp. 712–717.
- [44] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Secur. Trust*, 2022, pp. 1–11.
- [45] X. Wang, S. Chen, and J. Su, "App-Net: A hybrid neural network for encrypted mobile traffic classification," in *IEEE INFOCOM Conf. Comput. Commun. Workshops*, 2020, pp. 424–429.
- [46] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè, "MIMETIC: Mobile encrypted traffic classification using multimodal deep learning," *Comput. Netw.*, vol. 165, 2019, Art. no. 106944.
- [47] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020.



Shenghong Li (Senior Member, IEEE) received the BS and MS degrees in electrical engineering from the Jilin University of Technology, Changchun, China, in 1993 and 1996, respectively, and the PhD degree in radio engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 1999. Since 1999, he has been with Shanghai Jiao Tong University, Shanghai, China, as a research fellow, associate professor, and professor, successively. His research interests include information security, signal and information processing, and artificial intelligence.



Hao Peng received the PhD degree in computer science and technology from Shanghai Jiaotong University, in 2012. He is currently a full professor with the School of Computer Science and Technology, Zhejiang Normal University. He has authored or coauthored more than 100 papers in prestigious journals and conferences. His research interests include AI security, IoT security, software and system security, data-driven security, and Big Data mining and analysis. He was the program committee for more than ten international conferences.



Hua Ding (Student Member, IEEE) received the BS degree from the Dalian University of Technology, Dalian, China, in 2022. She is currently working toward the PhD degree with the School of Computer Science, Shanghai Jiao Tong University, Shanghai, China. Her research interests include encrypted network service, network security, and machine learning for networks.



Lixing Chen received the BS and ME degrees from the College of Information and Control Engineering, China University of Petroleum, Qingdao, China, in 2013 and 2016, respectively, and the PhD degree in electrical and computer engineering from the University of Miami, in 2020. He is currently a tenure-track assistant professor with the School of Computer Science, Shanghai Jiao Tong University, China. His research interests include mobile edge computing and machine learning for networks.



Zhe Qu (Member, IEEE) received the BS degree in automation from Xiamen University, Xiamen, China, in 2015, the MS degree in electrical and computer engineering from the University of Delaware, Newark, DE, USA, in 2017, and the PhD degree in electrical engineering from the University of South Florida, Tampa, FL, USA, in 2022. He is currently a tenure-track associate professor with the School of Computer Science and Engineering, Central South University, Changsha, China. His research interests include network and mobile system security, and machine learning for networks.



Bo Zhang received the BS degree from Hohai University, in 2007, the MSc degree from Southeast University, Nanjing, China, in 2012, and the PhD degree from the Nanjing University of Science & Technology, Nanjing, China, in 2018. He was a postdoc with Shanghai Jiao Tong University, in 2023 and 2025, respectively. His research interests include cybersecurity in smart grid and network security situation awareness.



Yang Bai (Member, IEEE) received the BS degree from Northeastern University, Shenyang, China, and the MS and PhD degrees from the College of Engineering, University of Miami, USA, in 2015 and 2021, respectively. She is currently an assistant professor with the Department of Automation, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, China. Her research interests include intelligent edge systems and industrial IoT.